CLOUD BASED BIG DATA PROCESSING AND PREDICTIVE ANALYTICS FOR 5G O RAN NETWORKS

Using the NetsLab 5G O RAN IDD Dataset and Amazon Web Services

FINAL PROJECT REPORT

Student Name: Alireza Eslamikhah
Student Number: 200550614

Course: Big Data Analytics & Cloud Computing
Term: Spring and Summer 2026

Abstract

This project investigates a cloud based approach for processing and classifying security data from a physical 5G Open Radio Access Network testbed. The work uses the NetsLab 5G O RAN intrusion detection dataset, which contains raw packet captures, structured packet records, network flow records, and lower layer radio measurements. The complete archive contains 320 files and about 135.93 GB of data. A focused Spark workload was selected from 35 packet level CSV files with 6.39 GB of structured data and 28,803,972 packet records. A temporary Amazon EC2 instance was used to transfer the required public data directly to Amazon S3. Amazon EMR Serverless then executed PySpark jobs for cleaning, aggregation, scenario analysis, and conversion to Snappy Parquet. Two binary Random Forest classifiers were trained with Spark MLlib. The network flow model used 1,723,817 records and achieved 97.66 percent test accuracy, 98.14 percent attack precision, 99.29 percent attack recall, and 98.71 percent attack F1. The radio model used 45,244 lower layer records and achieved 98.90 percent accuracy and 99.29 percent attack F1. The results show that a small serverless Spark configuration can process tens of millions of packet records and support machine learning without maintaining a permanent cluster. The study also identifies limits in class balance and time alignment between packet captures and radio telemetry, which are important for future cross layer experiments.

Keywords: 5G, O RAN, intrusion detection, Big Data, Apache Spark, Amazon EMR Serverless, Amazon S3, Random Forest, radio telemetry

All numerical results in this report were taken from the project output files produced during local profiling and AWS execution. Public references are used only for background and comparison. Where a source is cited in the report text, a short direct quotation is retained from that source.

1. Introduction

1.1 Motivation and Security Problem

Fifth generation mobile systems support a large number of users, services, devices, and software functions. This scale is useful for communication, but it also creates more places where abnormal behavior can appear. A security monitoring system therefore has to examine traffic quickly enough to separate normal activity from behavior that may indicate an attack. The distinction between benign and attack traffic is important because a network operator cannot treat every unusual packet as malicious. A wrong alarm wastes time and can lead to unnecessary blocking, while a missed attack can allow service degradation, data exposure, or continued reconnaissance.

Anomaly detection is especially useful in a mobile network because the traffic pattern is not fixed. Users enter and leave the network, applications change their sending rate, radio quality changes, and the same service can create different packet patterns at different times. The detection problem is therefore not only a question of finding a single bad packet. It is a question of identifying whether a set of measurements looks consistent with a benign network state or with an abnormal state. This project treats that decision as a binary classification problem, where label 0 represents benign traffic and label 1 represents attack traffic.

The practical cost of errors is not symmetric. False positives can interrupt legitimate users and create alert fatigue. False negatives are more dangerous because an attack is accepted as benign. For this reason, the evaluation in this project reports accuracy, precision, recall, F1 score, ROC AUC, and the complete confusion matrix rather than relying on accuracy alone. Recall for the

attack class is particularly important because it shows how many attack records were detected.

1.2 5G and O RAN Architecture

3GPP describes the 5G core by stating that *"The 5GC architecture relies on a "Service-Based Architecture" (SBA) framework"* [1]. This design separates major core functions into network services that communicate through defined interfaces.

At a high level, a 5G system contains user equipment, a radio access network, and the 5G core. The radio access network carries the radio connection between the user equipment and the rest of the network. O RAN adds more open and programmable interfaces to this part of the system. A traditional baseband function is separated into logical components such as the Central Unit, Distributed Unit, and Radio Unit. This separation creates useful observation points. The Central Unit can provide packet and network flow information, while the Distributed Unit can expose lower layer radio and MAC measurements.

The O RAN Alliance defines the E2 relationship in a way that can *"enable near-real-time control and optimization of RAN elements and resources via fine-grained data collection and actions"* [2]. The Near RT RIC can host xApps that use such information for control or monitoring tasks.

This open structure creates a security opportunity and a security challenge. The opportunity is that data can be collected from more than one layer. A detector can observe both traffic behavior and radio conditions. The challenge is that more software components, interfaces, and external applications increase the system surface that must be protected. The project therefore studies network flow data and radio telemetry as two separate but related views of the same general security problem.

1.3 Attack Types Considered

The dataset contains several attack families. Denial of Service traffic attempts to reduce availability by sending traffic that consumes bandwidth, connection state, or server resources. In this project the DoS family includes SYN, ACK, UDP, ICMP, HTTP flood, and Slowloris behavior. Distributed Denial of Service uses a similar goal but involves traffic from multiple sources. Probe activity includes port scanning and operating system fingerprinting. Brute force activity repeatedly tests credentials or login combinations, mainly over SSH and FTP in this dataset. Web attacks include HTTP brute force, directory brute force, SQL injection, and cross site scripting.

These attack types are not equally represented. Some flood scenarios produce millions of packet records, while web attacks and probe scenarios can be much smaller. This imbalance is expected in controlled security data but it affects evaluation. A model can achieve a high overall accuracy if it performs very well on a large attack class while still making mistakes on a smaller class. The project addresses this by keeping the primary prediction binary and by reporting class specific precision and recall.

1.4 Random Forest for Binary Detection

Breiman introduced the method with the statement that *"Random forests are a combination of tree predictors"* [8]. In practice, many decision trees are trained with random sampling of records and features, and their outputs are combined to make a final decision.

Random Forest was selected because it is a strong baseline for mixed network features, it can model nonlinear relations, and it requires less tuning than many deep learning methods. It also fits the course goal because Spark MLlib can train the model in the same distributed environment used for Big Data processing. The final implementation evaluates two forest sizes for each modality and selects the better setting on a validation set before measuring the untouched test set.

1.5 Project Objectives and Contributions

The project has four main objectives. First, it measures the real size and structure of the selected repository instead of describing the data only from documentation. Second, it moves a representative structured packet workload to cloud storage and processes the full 35 file group with Spark. Third, it trains separate network flow and radio Random Forest models using the complete prepared datasets. Fourth, it evaluates practical cloud choices such as temporary compute, serverless Spark, object storage, IAM roles, and automatic shutdown.

TABLE I

PROJECT OBJECTIVES AND COMPLETED IMPLEMENTATION

Objective	**Implemented work**	**Main evidence**
Understand the data	Profiled 320 files, formats, schemas, labels, and databases	135.93 GB repository inventory
Demonstrate Big Data processing	Processed all 35 packet CSV files with Spark	28,803,972 packet rows
Build attack classifiers	Trained network flow and radio Random Forest models	Two final test evaluations
Evaluate cloud value	Used EC2, S3, IAM, and EMR Serverless with automatic cleanup	Repeatable cloud pipeline and short lived compute

2. Related Work

2.1 5G Intrusion Detection Datasets

The NetsLab dataset descriptor presents the repository as *"a comprehensive dataset collected from a live 5G O-RAN testbed"* [3]. The importance of this dataset for the present project is its combination of packet traffic and radio telemetry from a physical O RAN environment.

The descriptor work is directly connected to this project because it explains how higher layer traffic and lower layer measurements can be collected in parallel. It also describes benign and attack experiments that cover DoS, DDoS, probe, brute force, and web attack families. The present project differs from the descriptor paper by focusing on a cloud data engineering pipeline, distributed Spark processing, Parquet conversion, and two practical Random Forest baselines.

Samarakoon et al. introduced 5G NIDD as *"a fully labeled dataset built on a functional 5G test network"* [4]. This earlier work showed the value of collecting attack data from a real 5G environment rather than using only generic enterprise intrusion datasets.

5G NIDD is mainly a network traffic dataset. It is useful for supervised intrusion detection and provides a strong example of how real test network data can support machine learning. The NetsLab

5G O RAN IDD repository extends the scope by providing radio telemetry in addition to network data. That extra modality is important because an O RAN security system may observe effects of attacks below the transport and application layers.

2.2 O RAN Radio and Cross Layer Detection

Dimou and Noubir report an O RAN sensing system with performance *"reaching 99.5% Accuracy with only 0.6% False Positives"* [5]. ARGOS uses richer user equipment telemetry and an xApp oriented design for real time rogue base station detection.

ARGOS demonstrates that radio information can support security monitoring without depending only on packet inspection. Its target attack and learning method are different from this project, but the work supports the decision to treat lower layer measurements as meaningful security features. In the current project, a supervised Random Forest is used instead of an unsupervised variational autoencoder because binary labels are already available.

A recent cross layer study on the same general O RAN data setting reports that *"Radio features match or outperform network flows on ROC-AUC and run-level detection rate across all architectures"* [6]. The same study also shows that fusion is not automatically better in every operating condition.

This result is important for the interpretation of the present work. It supports evaluating radio and network modalities independently before forcing them into a combined feature table. It also motivates stronger evaluation protocols such as run disjoint splitting. The current project uses a random train, validation, and test split to provide a clear course baseline, while run disjoint and temporal evaluation are identified as future work.

2.3 More Complex Machine Learning Models

Ilias et al. report that their convolutional and mixture of experts method *"reaches weighted F1-score up to 99.95%"* [7]. This shows that deep models can obtain very high scores on labeled 5G intrusion data.

A deep model may improve representation capacity, but it also increases training cost, implementation complexity, and the number of choices that have to be justified. This project deliberately uses Random Forest as a simpler model because the main course focus is Big Data analytics and cloud computing rather than model architecture search. The final results are therefore treated as scalable baselines, not as a claim that Random Forest is the best possible intrusion detector.

2.4 Research Gap Addressed by This Project

The reviewed studies mainly focus on dataset construction, intrusion detection accuracy, radio based sensing, or advanced model design. The gap addressed here is the complete engineering path from a large multi format public repository to a controlled AWS data lake, distributed Spark transformation, Parquet storage, and Spark MLlib evaluation. The project also records practical issues that are easy to miss in model centered studies, including a 135 GB source archive, local Windows Hadoop output problems, incomplete time alignment between some packet and radio files, class imbalance, cloud IAM design, and resource limits.

This engineering view is useful because a high model score alone does not show whether the data can be moved, organized, processed, reproduced, and controlled at reasonable cost. The project therefore evaluates both analytics quality and the data platform that produces the analytics.

3. Dataset and Big Data Characterization

3.1 Dataset Source and Testbed Context

The selected source is the NetsLab 5G Open Radio Access Network Intrusion Detection Dataset. The proposal selected this repository because it contains both higher layer traffic and lower layer radio information from a physical testbed. The local archive used for profiling contained 320 files. The final cloud experiment did not attempt to process every raw capture with Spark. Instead, it retained the full repository as the scale context, inspected the raw packet capture layer, and selected the largest consistent structured packet schema for distributed processing.

The repository contains benign experiments and controlled attack scenarios. The network layer includes raw packet captures and prepared flow records. The radio layer includes measurements such as RSRP, RSRQ, RSSI, SINR, CQI, uplink and downlink MCS, BLER, power related fields, user equipment identifiers, and cell information. The lower layer database contains timestamped records from May 9, 2025 to July 1, 2025.

3.2 Repository Inventory and Storage Volume

TABLE II

REPOSITORY INVENTORY FROM LOCAL PROFILING

Format	Files	Size	Role
PCAP	83	102.01 GB	Raw packet captures
PCAPNG	35	27.07 GB	Raw packet captures
CSV	110	6.62 GB	Packet, flow, and radio tables
Database	4	0.22 GB	Prepared SQLite data
TXT	49	0.02 GB	Lower layer telemetry records
JSON	39	about 0.00 GB	Metadata and supporting records

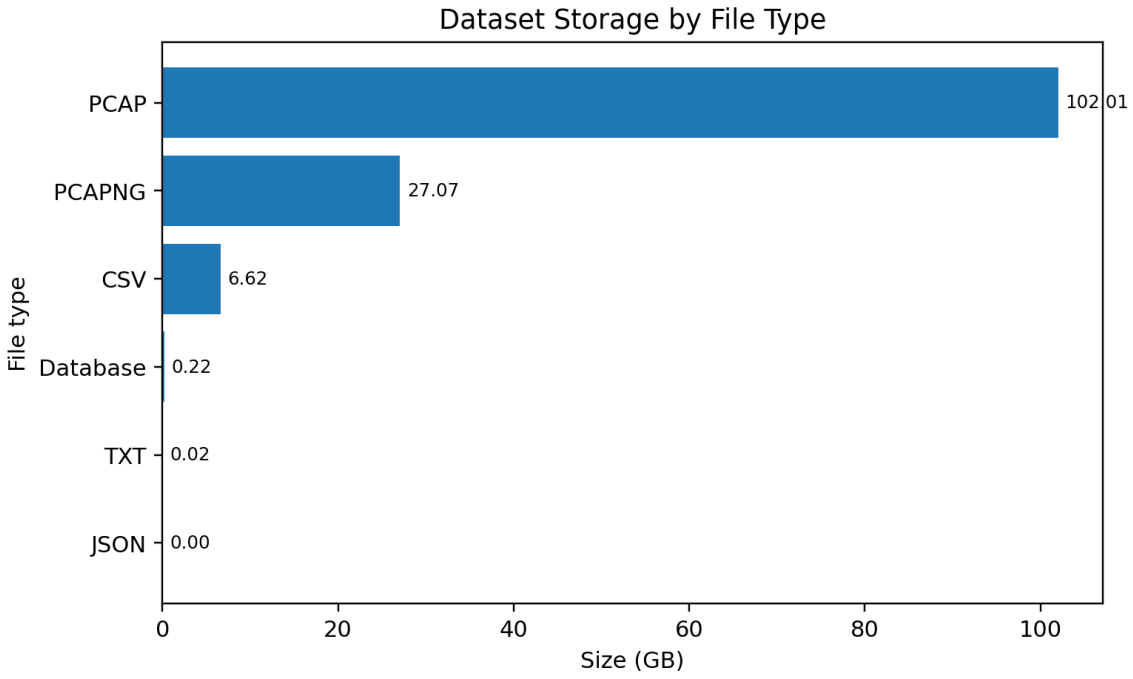


Fig. 1. Storage volume by file format. Raw PCAP and PCAPNG data account for most of the repository.

The total repository size was 135.93 GB. PCAP and PCAPNG files together accounted for about 129.08 GB, which means most bytes were stored in raw capture form. The structured CSV layer was much smaller at 6.62 GB, but it was still large enough to create a meaningful Spark workload. The largest consistent packet schema contained 35 CSV files, about 6.39 GB, and later produced 28,803,972 packet rows in the cloud job.

3.3 Network Flow Dataset

The prepared network database contains 1,723,817 flow records and 26 fields. Important fields include source and destination ports, protocol, service, duration, byte counts, packet counts, connection state, IP protocol, HTTP related indicators, attack category, attack type, and the binary traffic type. Identifier fields such as uid and IP addresses were useful for inspection but were excluded from the final model to reduce memorization and direct identity dependence.

TABLE III

NETWORK FLOW CLASS AND ATTACK CATEGORY DISTRIBUTION

Category	Records	Share
Attack traffic	1552952	90.09%
Benign traffic	170865	9.91%
DoS	632507	36.69%
DDoS	420282	24.38%
Web	288304	16.72%
Probe	183293	10.63%
Brute force	28566	1.66%

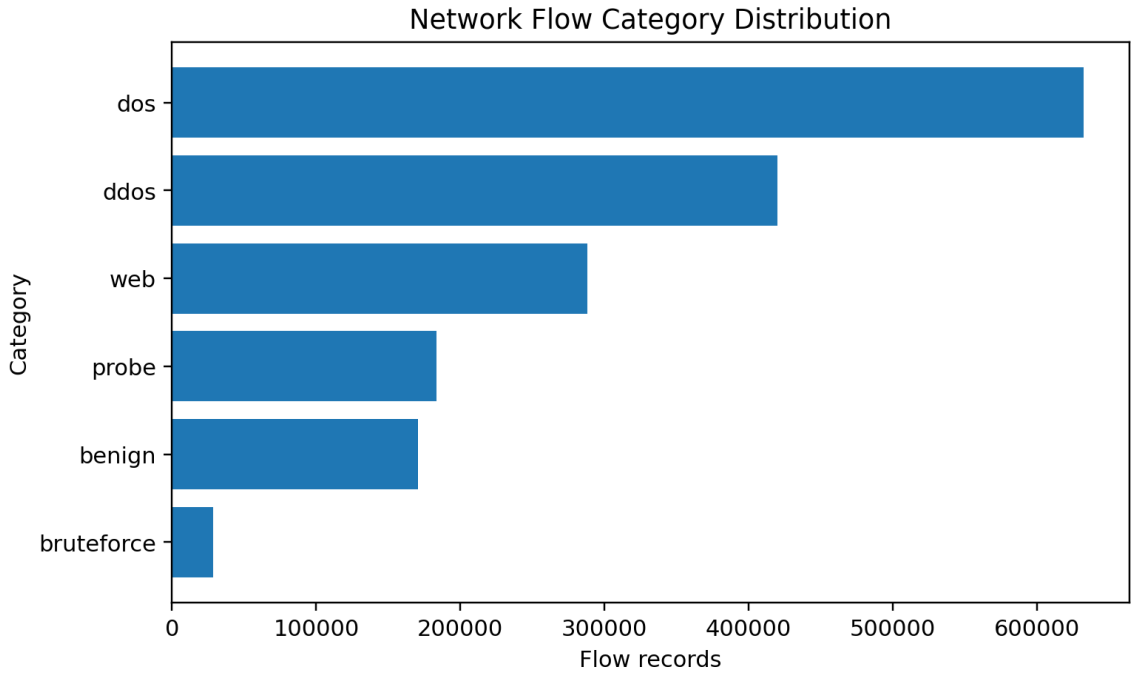


Fig. 2. Network flow category distribution. Attack records are much more common than benign records.

The binary label is highly imbalanced. About 90.09 percent of flow records are attacks and 9.91 percent are benign. The attack type distribution is also uneven. SYN is the largest named attack type with 506,451 records, followed by HTTP flood with 254,314, UDP with 227,408, and TCP ACK with 199,304. Small classes such as XSS and SQL injection have only 3,661 and 1,763 records. Because the final task is binary, these subcategories are not separately predicted, but the imbalance remains important when interpreting the confusion matrix.

3.4 Radio Telemetry Dataset

The lower layer database contains 45,244 records. The binary labels include 34,580 attack records and 10,664 benign records. The largest radio attack category is DDoS with 15,084 records, followed by Probe with 8,445, DoS with 4,721, Web Attacks with 3,273, and BruteForce with 3,057. The radio records include 5G signal and MAC information rather than only transport traffic.

TABLE IV

RADIO MEASUREMENTS USED BY THE FINAL RANDOM FOREST

Group	Features	Interpretation
Downlink	dlBytes, dlMcs, dlBler	Traffic volume, modulation setting, block errors
Uplink	ulBytes, ulMcs, ulBler	Traffic volume, modulation setting, block errors
Signal quality	rsrq, sinr, rsrp, rssi, cqi	Signal strength, quality, interference, channel condition

Group	Features	Interpretation
Power and link state	ri, phr, pcmax, pucchSnr, puschSnr	Rank, power headroom, power limit, uplink SNR

The final radio model uses 16 numeric measurements. Fields that can identify a run or device, including timestamp, user equipment ID, cell ID, RNTI, PMI, attack category, and attack subcategory, were not used as predictors. This reduces direct label leakage and keeps the classifier focused on radio behavior.

3.5 Why the Dataset Is Big Data

The project uses the three V view of Big Data: volume, variety, and velocity. A fourth practical issue, veracity, is also useful because security datasets may contain imbalance, irregular timing, or incomplete alignment. The dataset clearly provides evidence for all of these dimensions, but the velocity claim must be stated carefully because the project works with an offline snapshot rather than a live stream.

TABLE V

BIG DATA CHARACTERISTICS OBSERVED IN THE PROJECT

Dimension	Evidence in this dataset	Project response
Volume	135.93 GB total; 129.08 GB raw captures; 28.80 million packet rows in the main Spark workload	Use selective structured processing, distributed Spark, and object storage
Variety	PCAP, PCAPNG, CSV, TXT, JSON, SQLite; packet, flow, and radio modalities	Profile schemas first and keep separate S3 prefixes for each data type
Velocity	Timestamped radio and packet measurements were generated during live testbed experiments	Preserve timestamps and design batch processing that can later be extended to streaming
Veracity	Strong class imbalance and incomplete time overlap for some cross layer file pairs	Use full confusion matrices, avoid forced joins, and document limitations

4. Methodology and Cloud Architecture

4.1 Overall Design

The methodology was designed as a reproducible pipeline with separate stages for profiling, cloud ingestion, distributed packet processing, model training, and result storage. Local analysis was used to understand the repository and choose a useful workload before cloud resources were created. This avoided sending the full 135.93 GB archive to Spark without first knowing which files had consistent schemas.

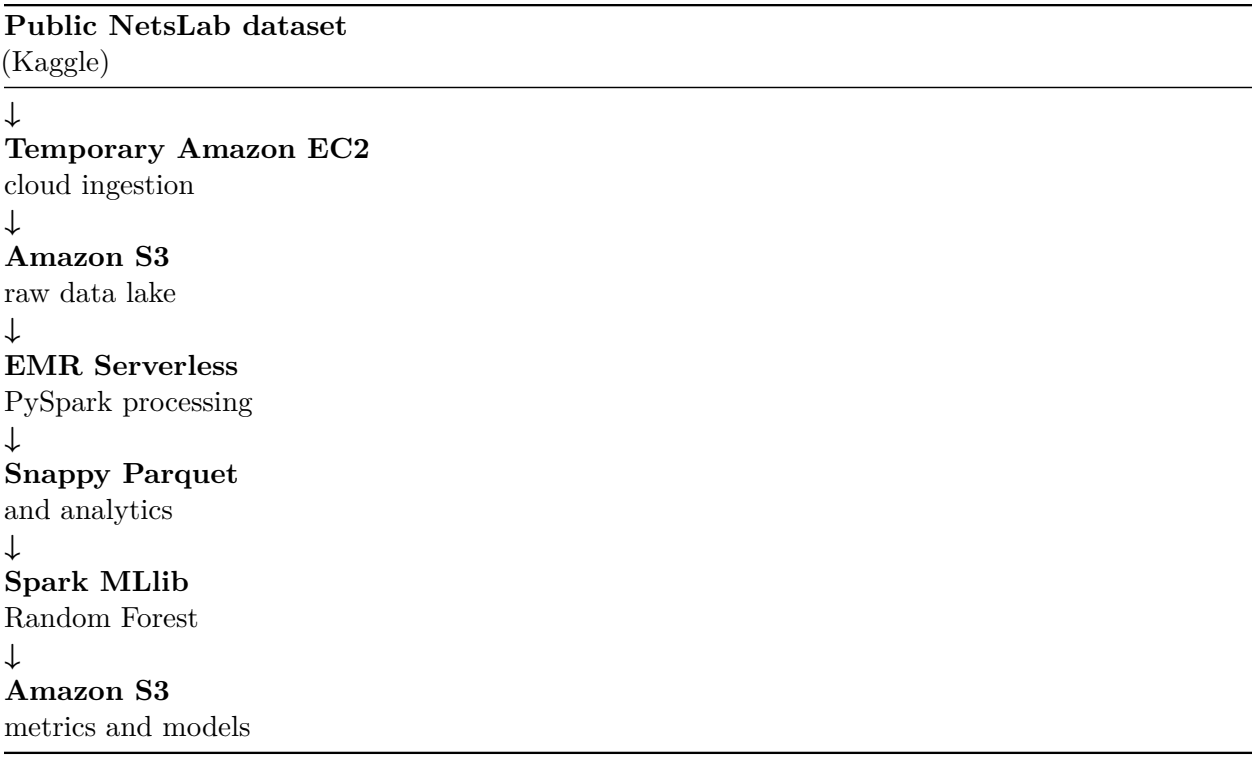


Fig. 3. Implemented cloud data and machine learning pipeline.

4.2 Local Profiling and Spark Feasibility Test

The first stage used Python to inventory all files, measure folder sizes, group file types, inspect the SQLite databases, count labels, and list schema fields. A second script grouped 110 CSV files by schema. The main packet schema contained 35 files and about 6.39 GB, representing 96.60 percent of all structured CSV bytes. This group was selected as the main Big Data workload because it was large, consistent, and directly related to the security scenarios.

A local PySpark test on Windows successfully read all 35 files and counted 28,803,972 records. It also completed packet scenario aggregation, but it failed when writing Spark output because the local Windows Hadoop environment did not have the expected HADOOP_HOME and winutils support. This was a useful engineering result. The computation itself worked, but local platform setup created avoidable friction. The cloud stage therefore moved the same logic to Linux based managed Spark instead of spending project time on Windows specific Hadoop configuration.

4.3 Cloud Data Ingestion with Temporary EC2

A temporary Amazon Linux 2023 EC2 instance was used only for cloud side ingestion. The instance type was t3.medium with a 60 GiB gp3 root volume. It used an IAM instance role that was limited to the project S3 bucket. Session Manager was enabled so the instance could be accessed through the AWS console without opening SSH to the public Internet. The ingestion script used KaggleHub to download only the required public files. Each packet CSV was uploaded to S3 and then deleted from the instance cache, which prevented the temporary disk from filling with the complete archive.

The ingestion run stored 35 packet CSV files, one prepared network flow CSV, and both the lower

layer SQLite database and a converted lower layer CSV. The S3 summary reported 6.4 GiB under raw/packets, 216.7 MiB under raw/network, and 11.7 MiB under raw/radio. After verification, the EC2 instance was terminated. This pattern kept the data transfer reproducible while ensuring that compute was not left running.

4.4 Amazon S3 Data Lake Organization

AWS defines the service simply: *"Amazon S3 is an object storage service"* [10]. The project uses S3 as the persistent layer for source data, processed tables, scripts, metrics, and trained models.

TABLE VI

PROJECT S3 ORGANIZATION

Prefix	Contents	Purpose
raw/packets/	35 packet CSV files	Main Spark processing input
raw/network/	Network_Dataset.csv	Network flow Random Forest input
raw/radio/	Lower_Layer_Data.db and CSV	Radio Random Forest input
processed/packets_parquet/	Scenario partitioned Snappy Parquet	Efficient structured packet output
scripts/	PySpark and ingestion scripts	Repeatable job definitions
results/bigdata_metrics/	Packet counts, timings, summaries	Big Data evaluation
results/network_model/	Metrics, confusion matrix, model, samples	Network ML outputs
results/radio_model/	Metrics, confusion matrix, model, samples	Radio ML outputs

Public access was blocked on the bucket. IAM policies were used instead of static access keys. One role supported the temporary EC2 ingestion instance, and a separate EMR runtime role granted the Spark jobs the minimum S3 access required for reading inputs and writing project outputs.

4.5 EMR Serverless Spark Environment

AWS explains the managed execution model by noting that *"you don't have to configure, optimize, secure, or operate clusters"* [11]. This was suitable for a short course project because the application could start for each job and stop again when idle.

The EMR Serverless application was created in the Canada West region, ca-west-1, using EMR release 7.13.0 and Spark 3.5.6 on x86_64. The application maximum was limited to 12 vCPU and 48 GB of memory. No preinitialized capacity was used. Auto start and auto stop were enabled, with a five minute idle timeout. Each final job used a driver and at most two executors. The Spark configuration set executor instances to 2, dynamic allocation minimum to 1, initial executors to 2, and maximum executors to 2. This kept the compute ceiling predictable.

The selected processing engine is appropriate because *"Apache Spark is a unified analytics engine for large-scale data processing"* [9]. PySpark allowed the same Python based workflow to run as distributed Spark jobs.

4.6 Packet Processing Pipeline

The packet job read all CSV files from the raw packet S3 prefix. Column names from Wireshark style exports were sanitized so names such as frame.len became frame_len. The source file path was retained and used to derive a controlled scenario label. The final label mapping contained ten expected scenarios: Benign; DDoS ICMP, SYN, TCP, and UDP; DoS ICMP, SYN, TCP, and UDP; and Probe PortScan. The script included a validation check so the job would fail if an unexpected scenario appeared or if an expected scenario was missing.

Important numeric fields were cast to double precision, including frame length, IP length, IP time to live, IP protocol, TCP length and ports, UDP length and ports, and ICMP type and code. A conservative cleaning rule retained rows with a valid positive frame length. The cleaned DataFrame was persisted with memory and disk storage because the job reused it for counts, aggregation, and Parquet output.

TABLE VII

PACKET SPARK PROCESSING STAGES

Stage	Operation	Output
Read	Load 35 CSV files from S3 and sanitize headers	Unified Spark DataFrame
Label	Add source path, source file, scenario, and protocol group	Scenario aware rows
Clean	Cast numeric fields and require positive frame length	28,803,972 valid rows
Aggregate	Count packets, sum bytes, calculate packet statistics and approximate unique IP counts	Scenario and protocol summaries
Transform	Repartition and write by scenario using Snappy Parquet	Processed packet data lake
Verify	Read the Parquet output and recount rows	28,803,972 verified rows

4.7 Parquet Transformation

Apache describes Parquet as *"an open source, column-oriented data file format designed for efficient data storage and retrieval"* [12]. The project uses Snappy compression and partitions the output by attack scenario.

Parquet was selected because later analytics usually read only a subset of columns and because the column oriented representation is a better fit for repeated analytical scans than raw CSV. The project writes 64 Spark partitions before scenario partitioning. The Parquet data is not treated as a replacement for the original files. The raw inputs remain in S3 while the processed Parquet layer acts as a prepared analytical representation.

4.8 Network Random Forest Method

The network model reads the full 1,723,817 row Network_Dataset.csv. The binary target is traffic_type. To reduce leakage, uid, source IP, destination IP, attack category, attack type, and

history are excluded. The categorical predictors are proto, service, and conn_state. These fields are indexed and one hot encoded. Numeric predictors include ports, duration, source and destination byte counts, missed bytes, packet counts, IP byte counts, IP protocol, HTTP depth, file byte count, GET method indicator, HTTP status error, and file transfer indicator. Missing numeric values are replaced with zero, while categorical indexers keep previously unseen values in an additional category.

The data is split with a fixed random seed into 70 percent training, 15 percent validation, and 15 percent test data. Two candidate forests are trained. Candidate 1 uses 60 trees with maximum depth 10. Candidate 2 uses 100 trees with maximum depth 12. Both use maximum bins 64 and the square root feature subset rule. The validation attack F1 score determines the selected model. Only after this choice is the untouched test split evaluated.

4.9 Radio Random Forest Method

The radio model follows the same general train, validation, and test workflow. The input contains 45,244 records. Sixteen radio measurements are used: dlBytes, dlMcs, dlBler, ulBytes, ulMcs, ulBler, ri, phr, pcmax, rsrq, sinr, rsrp, rssi, cqi, pucchSnr, and puschSnr. Timestamp, user equipment ID, cell ID, RNTI, PMI, attack category, and attack subcategory are not used as predictors. This makes the result easier to interpret as a radio behavior model rather than a memorization of experiment identity.

Candidate 1 uses 60 trees with maximum depth 8 and 32 bins. Candidate 2 uses 100 trees with maximum depth 12 and 32 bins. The model with the better validation attack F1 is tested on the final holdout set. The trained Spark pipeline, validation table, test metrics, confusion matrix, feature importance, and a small prediction sample are written back to S3.

4.10 Evaluation Metrics

The evaluation uses the attack class as the positive class. True positives are attack records correctly detected as attack. True negatives are benign records correctly detected as benign. False positives are benign records classified as attack, and false negatives are attack records classified as benign. The main metrics are calculated as follows:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

$$F1 = 2 \times Precision \times Recall / (Precision + Recall)$$

ROC AUC is also reported because it summarizes the ranking quality across many decision thresholds. The confusion matrix remains essential because it shows the actual number of benign and attack mistakes. Given the class imbalance, a high accuracy without a confusion matrix would not be sufficient evidence.

5. Evaluation and Results

5.1 Packet Data Processing Results

The final cloud Spark job discovered all 35 packet CSV files and processed 28,803,972 rows. The cleaning rule retained every row, so the clean count remained 28,803,972 and zero rows were removed. This does not mean that every field was complete. It means that every row satisfied the minimum packet validity rule used by this stage, a non null positive frame length.

TABLE VIII

MEASURED PACKET PIPELINE METRICS ON EMR SERVERLESS

Metric	Result
Input files	35
Input rows	28,803,972
Clean rows	28,803,972
Removed rows	0
CSV count time	11.668 s
Clean and cache time	258.175 s
Analytics time	8.300 s
Parquet write time	56.762 s
Parquet read and count time	2.067 s
Verified Parquet rows	28,803,972

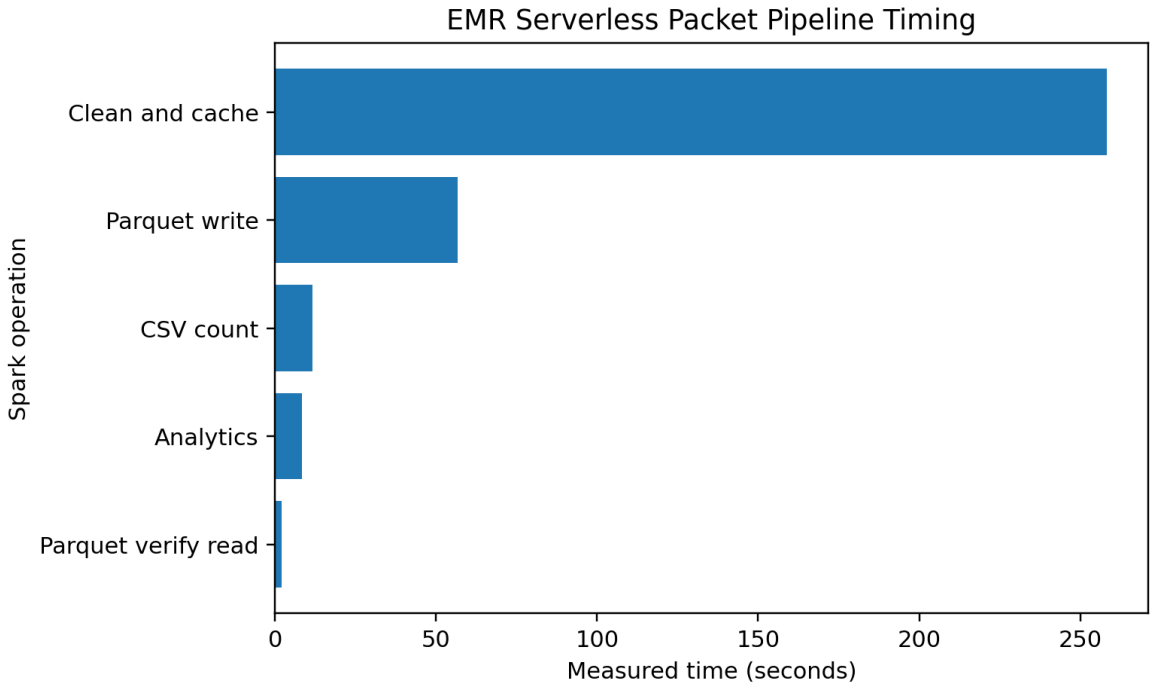


Fig. 4. Measured times for the major packet processing operations on EMR Serverless.

The longest measured phase was the clean and cache materialization step at 258.175 seconds. This phase triggers a complete Spark action while filling the persisted representation. The Parquet

write required 56.762 seconds. The final read and count of the prepared Parquet output took 2.067 seconds. These values should not be interpreted as a controlled storage benchmark because different operations and cache states are involved. They do show that the prepared Parquet layer could be read and verified very quickly after it was created.

5.2 Packet Scenario Analysis

TABLE IX

PACKET SCENARIO DISTRIBUTION FROM THE FULL SPARK WORKLOAD

Scenario	Packets	Share	Average bytes
DoS_SYN	11,661,285	40.48%	100.0
DoS_ICMP	6,764,757	23.49%	385.3
DoS_UDP	2,988,005	10.37%	1085.3
DDoS_SYN	2,609,555	9.06%	77.9
Benign	1,606,805	5.58%	1884.0
DDoS_UDP	1,441,650	5.01%	63.4
DDoS_TCP	867,026	3.01%	95.0
DDoS_ICMP	634,548	2.20%	88.0
DoS_TCP	198,876	0.69%	84.2
Probe_PortScan	31,465	0.11%	80.7

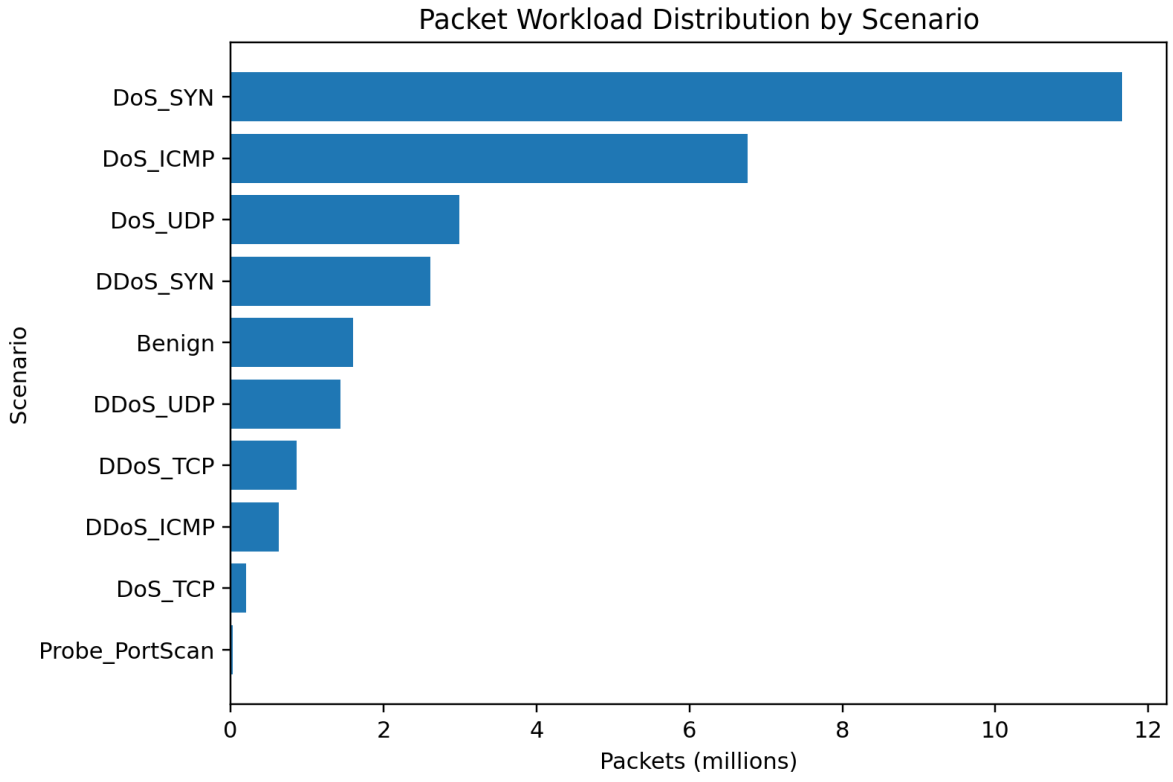


Fig. 5. Packet counts by scenario for the 28.80 million record structured workload.

DoS SYN is the dominant packet scenario with 11,661,285 records, or about 40.48 percent of the packet workload. DoS ICMP contributes another 6,764,757 records. Benign traffic contains 1,606,805 packet records, about 5.58 percent of this structured packet group. Probe PortScan is the smallest scenario at 31,465 packets. This distribution confirms that packet level security data is strongly imbalanced and that raw accuracy would be a poor choice for a multiclass model without additional treatment.

Packet size also varies substantially by scenario. The benign group has an average packet size of about 1,883.96 bytes in the extracted records, while DoS SYN averages 100 bytes and DDoS UDP averages about 63.43 bytes. These differences show that simple packet statistics carry useful scenario information, although a robust detector should not rely on size alone because traffic tools and network conditions can change.

5.3 Network Flow Random Forest Validation

TABLE X

NETWORK RANDOM FOREST VALIDATION CANDIDATES

ID	Trees	Depth	Fit s	Accuracy	Precision	Recall	F1	ROC AUC
1	60	10	45.40	0.9762	0.9807	0.9931	0.9869	0.9950
2	100	12	102.20	0.9768	0.9811	0.9934	0.9872	0.9956

The 100 tree model produced the better validation result. Its attack F1 score was 0.9872 compared with 0.9869 for the 60 tree model, and its ROC AUC was 0.9956 compared with 0.9950. The gain is small, but the larger model was selected according to the predefined validation rule rather than by looking at the final test data.

5.4 Network Flow Random Forest Test Results

TABLE XI

FINAL NETWORK FLOW TEST PERFORMANCE

Measure	Value
Full input rows	1,723,817
Training rows	1,206,950
Validation rows	258,144
Test rows	258,723
Accuracy	0.9766
Attack precision	0.9814
Attack recall	0.9929
Attack F1	0.9871
ROC AUC	0.9956
True negatives	21,147
False positives	4,395
False negatives	1,653

Measure	Value
True positives	231,528
Test evaluation time	5.607 s
Total model job logic time	197.351 s

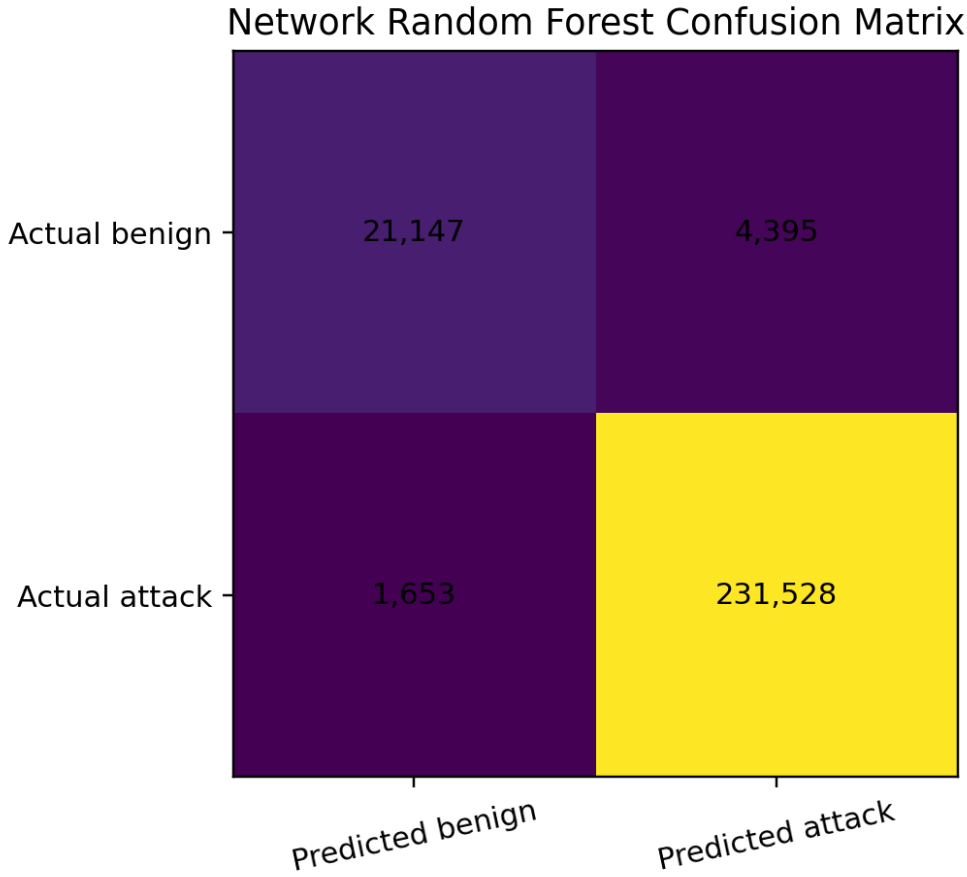


Fig. 6. Confusion matrix for the final network flow Random Forest test split.

The final network model correctly detected 231,528 attack records and missed 1,653 attacks. This gives an attack recall of about 99.29 percent. The model produced 4,395 false alarms among benign records. Because the test split contains far more attacks than benign records, the 97.66 percent accuracy should be read together with the 98.14 percent attack precision and the confusion matrix.

The network result is close to the earlier local baseline but is more useful for the final report because it uses the full 1.72 million record dataset in Spark, includes a separate validation set, and records an untouched final test result. The model also avoids direct attack category and attack type fields, which would make the binary prediction unrealistically easy.

5.5 Radio Random Forest Validation

TABLE XII

RADIO RANDOM FOREST VALIDATION CANDIDATES

ID	Trees	Depth	Fit s	Accuracy	Precision	Recall	F1	ROC AUC
1	60	8	6.02	0.9721	0.9653	0.9994	0.9820	0.9967
2	100	12	9.78	0.9911	0.9895	0.9988	0.9942	0.9994

The radio validation result shows a clearer difference between the two candidates. The 60 tree model achieved 0.9820 attack F1 and 0.9967 ROC AUC. The 100 tree model improved to 0.9942 F1 and 0.9994 ROC AUC, so it was selected for the final radio test.

5.6 Radio Random Forest Test Results

TABLE XIII

FINAL RADIO TEST PERFORMANCE

Measure	Value
Full input rows	45,244
Benign rows	10,664
Attack rows	34,580
Training rows	31,829
Validation rows	6,707
Test rows	6,708
Accuracy	0.9890
Attack precision	0.9877
Attack recall	0.9981
Attack F1	0.9929
ROC AUC	0.9990
True negatives	1,486
False positives	64
False negatives	10
True positives	5,148
Test evaluation time	2.115 s
Total model job logic time	37.386 s

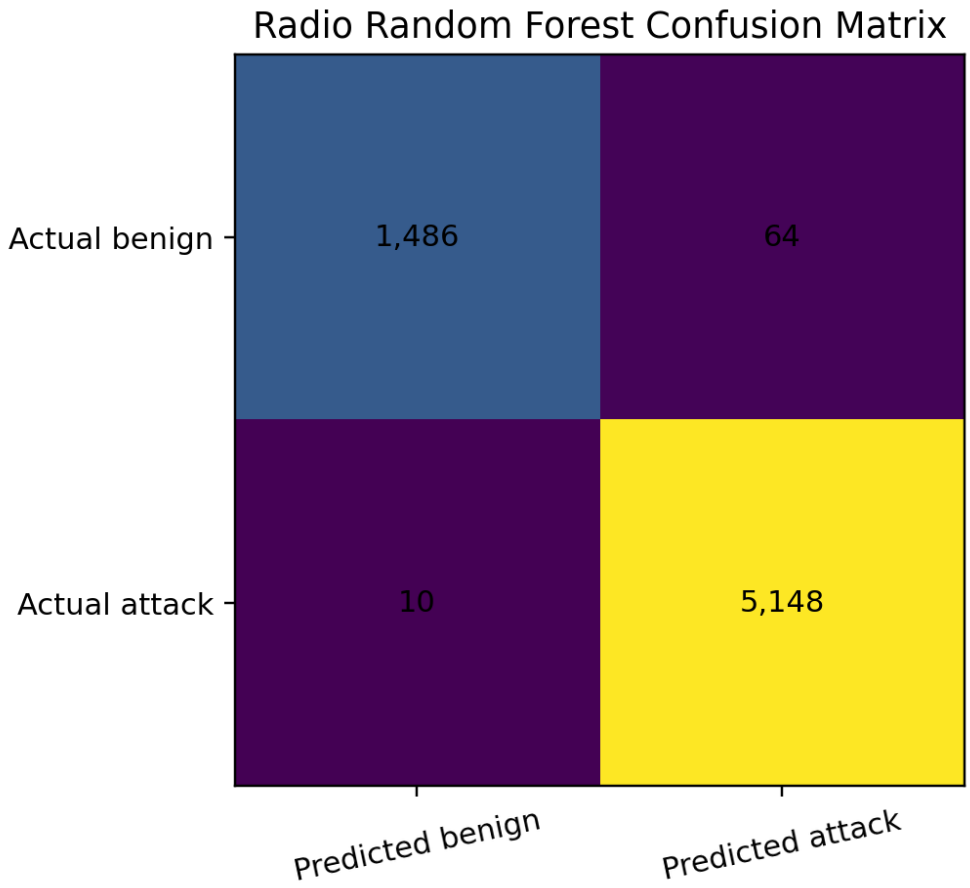


Fig. 7. Confusion matrix for the final radio Random Forest test split.

The radio model detected 5,148 of 5,158 attack records in the test split, missing only 10. It produced 64 false positives among 1,550 benign test records. The resulting attack recall is about 99.81 percent and the attack F1 score is about 99.29 percent. This result demonstrates that the selected radio measurements contain strong information about the controlled attack conditions in the dataset.

The radio score is very high, so it should be interpreted carefully. The split is random at the row level. Records from the same experiment type can therefore appear in training and test data even though direct identifiers are removed. This can make the test distribution easier than a future deployment where an entirely new run, day, user, or attack variation is encountered.

5.7 Comparison of the Two Classifiers

TABLE XIV

FINAL NETWORK AND RADIO MODEL COMPARISON

Metric	Network flow	Radio
Accuracy	0.9766	0.9890
Attack precision	0.9814	0.9877
Attack recall	0.9929	0.9981
Attack F1	0.9871	0.9929

Metric	Network flow	Radio
ROC AUC	0.9956	0.9990
Input rows	1,723,817	45,244
Total job logic time	197.351 s	37.386 s

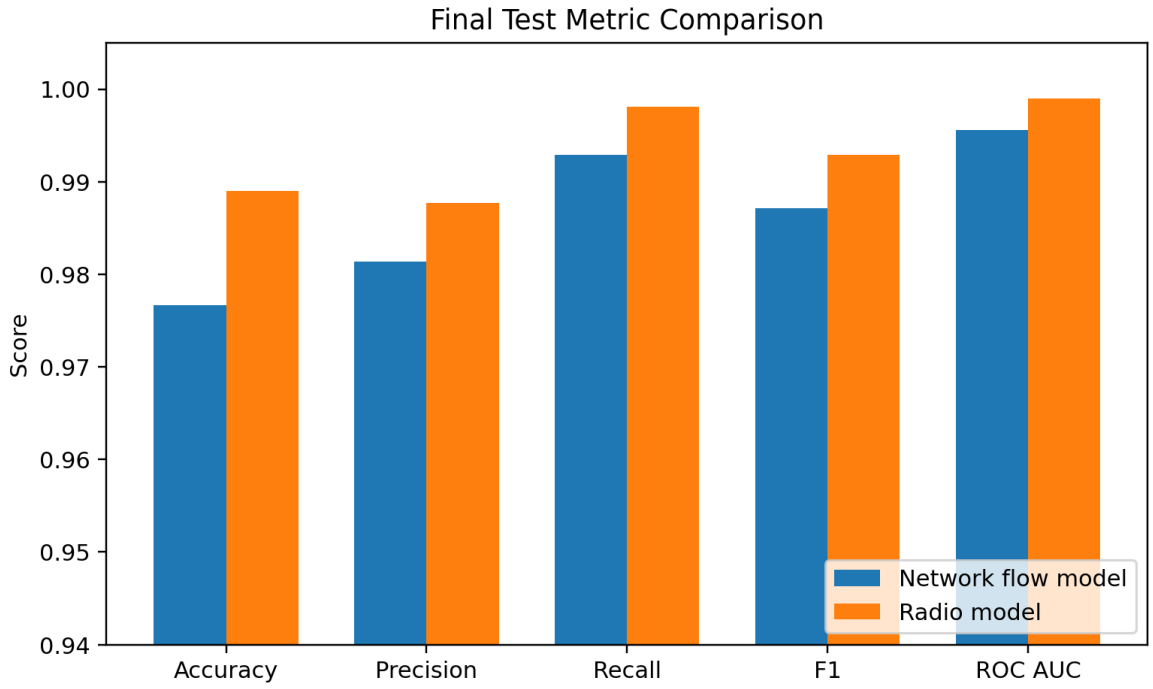


Fig. 8. Comparison of final network flow and radio test metrics.

The radio model has the higher score on every reported test metric, while the network model operates on a dataset that is about 38 times larger. This does not prove that radio data is universally better. The two datasets have different sizes, feature definitions, experiment structure, and class balance. The comparison is most useful as evidence that both modalities support binary detection and that the radio layer deserves separate attention in O RAN security research.

5.8 Cloud Processing Versus Local Practical Experience

The local Spark test confirmed that a single machine could read and aggregate the 6.39 GB structured packet workload, but the Windows environment failed during Spark output because native Hadoop support was not configured. The EMR Serverless run completed the same broader workflow, including S3 input, full row validation, aggregation, Parquet output, and output verification. This difference is not a proof that cloud computing is always faster. It shows that a managed Linux Spark environment reduced platform setup effort and made the output pipeline reproducible.

The cloud setup also allowed independent jobs to reuse the same S3 data lake. The packet processing job, network Random Forest job, and radio Random Forest job all used one EMR Serverless application and separate scripts. No permanent Spark cluster was maintained. After each experiment, workers were released and the application automatically stopped after the configured idle period.

6. Discussion and Limitations

6.1 What the Results Show

The project demonstrates that a structured subset of a much larger 5G security repository can be processed as a real distributed workload without claiming that every 135.93 GB was processed by Spark. This distinction is important. The complete repository establishes the source scale and variety. The actual Spark computation covers 6.39 GB of structured packet CSV data and 28.80 million records, which is the workload used for measured distributed analytics.

The machine learning results show strong binary separation for both network flow and radio data. The network model has very high attack recall, which is useful for reducing missed attacks, but it also creates more false positives than false negatives. The radio model has fewer errors in the final split. Both models therefore support the basic project question of whether these data layers contain enough information to distinguish benign and attack states.

6.2 Class Imbalance

Class imbalance is present at several levels. In the network flow dataset, attacks account for about 90.09 percent of all rows. The packet workload is also dominated by a few flood scenarios. This can make accuracy appear stronger than practical benign performance. The report therefore uses precision, recall, F1, ROC AUC, and confusion matrices. A future system should also study balanced training weights, per scenario recall, precision recall curves, and operating thresholds that are selected according to the cost of false alarms.

6.3 Random Split Limitation

Both final models use a deterministic random split. This is a reasonable baseline because it provides reproducible training, validation, and test sets. However, it is not the strongest possible generalization test. Neighboring records or records from the same attack run can be similar. A stronger evaluation would keep complete experiment runs, days, or user equipment sessions out of training. This is especially important for the radio result, where very high scores may partly reflect stable conditions within the controlled testbed.

The recent cross layer literature uses run disjoint evaluation, and that design should be adopted in a follow up study. The present report does not hide this limitation. The reported test scores measure generalization to held out rows drawn from the same repository, not guaranteed performance on a new operator network.

6.4 Cross Layer Fusion Limitation

The proposal originally considered a combined feature table. The timestamp study showed that some packet and radio files can be aligned, but not all selected scenarios have reliable overlap. Most importantly, the selected benign packet capture did not overlap with its lower layer telemetry. Training a combined benign versus attack model with only aligned attack examples would not be meaningful. The final design therefore uses two separate classifiers and treats data fusion as future work.

6.5 Big Data Scope and Streaming

The project is a batch analytics project. It does not claim to implement a live streaming intrusion detector. The source testbed generated time ordered network and radio observations, which demonstrates the velocity dimension of the data source, but the cloud pipeline processes an offline snapshot. Kafka and continuous inference were left out so the project could focus on reliable batch processing, cloud storage, and measured model evaluation. A streaming extension would require event time handling, window definitions, late data policy, and online model serving.

6.6 Cost and Security Considerations

The AWS design was intentionally small. The data ingestion server was temporary and was terminated after transfer. The Spark application had a fixed maximum capacity, no preinitialized workers, and an automatic idle stop. S3 was used for persistent storage while compute remained short lived. IAM roles were scoped to the project bucket, and the ingestion instance was accessed with Session Manager rather than an open SSH rule. A monthly budget was configured before computation began.

The report does not treat the exact billing screen as a scientific measurement because AWS billing estimates can update after usage is processed. The more important engineering result is that the architecture contains explicit controls for resource limits and cleanup. This makes it suitable for a student project and provides a clear path to scale the same jobs with more workers if a larger workload is required.

TABLE XV

MAIN LIMITATIONS AND FUTURE MITIGATION

Limitation	Effect	Recommended next step
Random row split	May overestimate performance on new experiment runs	Use run disjoint and time ordered test sets
Class imbalance	Accuracy can hide benign errors	Use class weighting and per class threshold analysis
Incomplete packet and radio alignment	Prevents reliable full cross layer fusion	Build a verified run map before joining modalities
Offline batch pipeline	Does not measure live detection delay	Add streaming event time windows and online inference
One testbed source	May not represent other operators or radio conditions	Validate on an independent O RAN environment
Limited model search	Does not compare all algorithms	Add Decision Tree, gradient boosting, and selected deep baselines

7. Reproducibility and Implementation Summary

7.1 Reproducible Artifacts

The implementation is separated into small scripts so each stage can be repeated independently. Local scripts profile the repository and group CSV schemas. The cloud ingestion script downloads

selected public files and places them in S3. The packet PySpark script processes the complete structured packet group and writes metrics. The network and radio MLlib scripts train candidate forests, select a model on validation F1, evaluate the final test set, and save the fitted Spark pipeline.

TABLE XVI

CORE IMPLEMENTATION ARTIFACTS

Script	Role
00_bigdata_profile.py	Repository and database scale summary
04_csv_schema_groups.py	CSV schema grouping and selection of the main packet workload
05_pyspark_packet_pipeline_fixed3.py	Local full data Spark feasibility test
ingest_selected_to_s3.py	Kaggle to EC2 to S3 ingestion
06_emr_serverless_packet_pipeline_v2.py	Final packet processing and Parquet transformation
07_emr_network_rf_v2.py	Final network flow Random Forest
08_emr_radio_rf.py	Final radio Random Forest

7.2 Data Lineage

The data lineage is simple enough to audit. Public dataset files are placed under raw S3 prefixes and are not overwritten by processing jobs. Derived packet data is written under processed/packets_parquet. Metrics and model outputs are written under results. The scripts are stored under scripts. This separation allows the input data, derived data, code, and results to be inspected independently.

The packet job writes a run metrics file containing the input file count, input row count, clean row count, measured operation times, and verified Parquet row count. The ML jobs write validation candidate tables, final metrics, confusion matrices, prediction samples, feature importance outputs, and fitted Spark pipelines. These small outputs make it possible to reproduce the tables in this report without reading AWS console screenshots.

7.3 Practical Lessons

Three implementation lessons were especially important. First, data inventory should come before cloud processing. The repository contained several representations of similar experiments, and processing everything would have increased cost without improving the main course objective. Second, scenario mapping should be validated explicitly. An early script version mapped several DDoS filenames incorrectly; adding a required scenario set prevented silent labeling errors in the final run. Third, output format matters. The Parquet layer gives a cleaner analytics interface than the original CSV collection and can be reused by later Spark jobs.

A fourth lesson concerns model engineering. The first network ML run trained successfully but failed near the end because Spark vector probabilities cannot be written directly to CSV. The corrected script converted the probability vector to a scalar attack probability before saving the prediction sample. The final rerun completed successfully. This type of failure shows why end to end validation is necessary even when the model itself trains correctly.

7.4 Final Project Answers

TABLE XVII

ANSWERS TO THE MAIN PROJECT QUESTIONS

Question	Answer from the completed project
How can the data be prepared?	Profile schemas first, process the consistent packet group with PySpark, and store derived data as scenario partitioned Parquet.
Can attack and benign states be predicted?	Yes. The final network flow model reached 0.9871 attack F1 and the radio model reached 0.9929 attack F1 on random holdout rows.
Does cloud processing add value?	Yes for reproducibility and managed Spark execution. It removed local Windows Hadoop output problems and allowed one S3 data lake to serve all jobs.
Can radio and network data simply be fused?	Not safely for all selected runs. Time alignment must be verified before a combined benign versus attack experiment.

8. Conclusion and Future Work

8.1 Conclusion

This project implemented a complete cloud based Big Data workflow for a 5G O RAN intrusion detection dataset. The source archive contains 320 files and 135.93 GB across raw packet captures, structured CSV data, telemetry text, JSON, and SQLite databases. Rather than claiming that the complete archive was processed with Spark, the project selected the largest consistent structured packet group, 35 CSV files and 6.39 GB, and processed all 28,803,972 records on Amazon EMR Serverless.

The processing pipeline cleaned and labeled the packet data, calculated distributed scenario statistics, wrote Snappy Parquet partitioned by scenario, and verified that the output contained the same 28,803,972 rows. The final Parquet verification read and count completed in 2.067 seconds after the output was prepared. The S3 data lake also supported full Spark MLlib experiments on 1,723,817 network flow records and 45,244 radio records.

The selected network Random Forest used 100 trees and maximum depth 12. It achieved 97.66 percent accuracy and 98.71 percent attack F1 on 258,723 test records. The radio Random Forest also used 100 trees and maximum depth 12. It achieved 98.90 percent accuracy and 99.29 percent attack F1 on 6,708 test records. These results show that both traffic behavior and radio measurements can support attack detection in the controlled dataset.

The main value of the work is not only the classifier score. The project shows how a large mixed repository can be profiled, reduced to a defensible analytical workload, transferred to cloud storage, processed with managed Spark, converted to a more efficient analytical format, and used for reproducible machine learning. It also documents important limits in class balance, random splitting, and cross layer time alignment.

8.2 Future Work

The first future step should be a run disjoint evaluation. Complete experiment runs should be held out so that the model is tested on conditions that are more independent from training. A time ordered split is another useful test for radio telemetry. These evaluations may lower the reported scores, but they would provide a better estimate of deployment performance.

The second step should be a verified cross layer feature table. Packet and radio files should be mapped by experiment identifier and exact UTC overlap before aggregation. Benign and attack runs must both be represented. Once alignment is reliable, radio features and network flow features can be compared as separate models and as a combined model under the same run disjoint split.

The third step is streaming. The current batch jobs could be extended with structured streaming or Kafka so that packet summaries and radio measurements are processed in event time windows. This would allow measurement of detection delay and would move the project from offline classification toward an operational intrusion detection workflow.

The final step is broader model comparison. A Decision Tree would provide a simple interpretability baseline. Gradient boosted trees could test whether the structured features benefit from a stronger tree ensemble. A carefully selected deep model could then be compared only after the data split is made more realistic. Model complexity should be added after the evaluation protocol is strong, not before.

References

[1] 3GPP, "5G System Overview," Aug. 2022, aug. 8, 2022. [Online]. Accessed: Aug. 11, 2026. [Online]. Available: https://www.3gpp.org/technologies/5g-system-overview

[2] O-RAN ALLIANCE, "WG3: Near-Real-time RAN Intelligent Controller and E2 Interface Workgroup," [Online]. Accessed: Aug. 11, 2026. [Online]. Available: https://www.o-ran.org/technical-groups/wg3

[3] F. A. Zadeh, A. Civciss, V. Ravihansa, C. Sandeepa, and M. Liyanage, "Descriptor: 5G Open Radio Access Network Multi-Modal Intrusion Detection Dataset (NetsLab-5GORAN-IDD)," *IEEE Data Descriptions*, vol. 2, pp. 348–357, 2025.

[4] S. Samarakoon, Y. Siriwardhana, P. Porambage, M. Liyanage, S.-Y. Chang, J. Kim, J. Kim, and M. Ylianttila, "5G-NIDD: A Comprehensive Network Intrusion Detection Dataset Generated over 5G Wireless Network," 2022. [Online]. Available: https://arxiv.org/abs/2212.01298

[5] S. Dimou and G. Noubir, "ARGOS: Anomaly Recognition and Guarding through O-RAN Sensing," 2025. [Online]. Available: https://arxiv.org/abs/2506.06916

[6] H. Fard, I. Komarov, and G. Wunder, "Cross-Layer Intrusion Detection in 5G O-RAN: Gains and Limits of Fusing Radio Telemetry with Network Flow Records," 2026, accepted for IEEE International Conference on Cyber Security and Resilience. [Online]. Available: https://arxiv.org/abs/2606.22450

[7] L. Ilias, G. Doukas, V. Lamprou, C. Ntanos, and D. Askounis, "Convolutional Neural Networks and Mixture of Experts for Intrusion Detection in 5G Networks and beyond," 2025, arXiv version 2. [Online]. Available: https://arxiv.org/abs/2412.03483

[8] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, Oct. 2001.

[9] Apache Software Foundation, "Apache Spark Documentation," [Online]. Accessed: Aug. 11, 2026. [Online]. Available: https://spark.apache.org/docs/latest/

[10] Amazon Web Services, "What is Amazon S3?" Amazon Simple Storage Service User Guide, [Online]. Accessed: Aug. 11, 2026. [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html

[11] ——, "What is Amazon EMR Serverless?" Amazon EMR Serverless User Guide, [Online]. Accessed: Aug. 11, 2026. [Online]. Available: https://docs.aws.amazon.com/emr/latest/EMR-Serverless-UserGuide/emr-serverless.html

[12] Apache Software Foundation, "Apache Parquet," [Online]. Accessed: Aug. 11, 2026. [Online]. Available: https://parquet.apache.org/